

EyesUp

Phase-Wise Build Plan for Google Antigravity

This plan turns the app concept — reading PDF or photo text aloud so a student never has to glance away from the notebook — into a sequence of build phases for Google Antigravity. Each phase is written as a task prompt you can hand to the agent, plus suitable technologies and checkpoints worth reviewing before moving on.

Phase 1 — Design System + Scaffold

- **Goal:** Establish the visual identity and app shell before any logic is added.

Agent task prompt

Scaffold a React + Vite app. Build the design tokens first — color palette, type pairing, spacing scale — per the brief: warm notebook-cream background, fountain-pen indigo primary, soft highlighter-yellow accent for the active sentence; a legible serif/handwriting-adjacent display face paired with a clean sans body face. Then build the shell: split-view layout (document/photo preview on one side, playback panel on the other), upload zone, and empty states.

Suitable technologies

- React + Vite — fast dev server, component-based UI
- Tailwind CSS — utility classes for quickly applying the token system
- Google Fonts (e.g. a serif display + Inter/Work Sans body) — type pairing

Checkpoints

- Review the scaffold visually before adding any logic.
 - Confirm the palette and type pairing don't default to a generic AI-tool look.
-

Phase 2 — OCR Pipeline

- **Goal:** Turn uploaded PDFs and photos into clean, extractable text.

Agent task prompt

Add PDF text extraction (PDF.js) and image OCR (Tesseract.js), with an animated, on-brand loading state (not a generic spinner) and clear error handling for blurry or scanned input.

Suitable technologies

- PDF.js — extracts selectable text directly from text-based PDFs
- Tesseract.js — in-browser OCR for photos and scanned/image-based PDFs
- pdf-lib or pdf.js canvas render — rasterizing scanned PDF pages before OCR

Checkpoints

- Test extraction accuracy against a real sample PDF and a real photo before moving on.
-

Phase 3 — Text-to-Speech + Live Highlighting

- **Goal:** Deliver the core feature: hands-free reading that keeps pace with writing.

Agent task prompt

Wire the Web Speech API for sentence-by-sentence reading. Sync a live text highlight (the signature 'now reading' effect) with speech boundaries. Add play/pause/speed/voice controls.

Suitable technologies

- Web Speech API (SpeechSynthesis) — free, built-in, no backend needed
- compromise.js or a simple regex sentence-splitter — reliable sentence chunking
- React state/useReducer — tracking current sentence index for highlight sync

Checkpoints

- This is the highest-polish phase — it's the feature that actually solves the problem.
 - Verify highlight sync stays accurate across long paragraphs.
-

Phase 4 — Hands-Free Interaction

- **Goal:** Make sure hands never need to leave the notebook or pen.

Agent task prompt

Add keyboard shortcuts (spacebar = play/pause, arrows = skip sentence / repeat last), large touch-friendly buttons, and a floating mini-control bar that stays visible while scrolling.

Suitable technologies

- native KeyboardEvent listeners — spacebar/arrow-key shortcuts
 - CSS position: sticky/fixed — persistent floating control bar
 - lucide-react — accessible, consistent icon set for controls
-

Phase 5 — Micro-Interactions & Motion

- **Goal:** Add one deliberate, delightful moment without cluttering the experience.

Agent task prompt

Add one signature motion moment — e.g. a page-load reveal or a satisfying transition when a new sentence starts highlighting. Keep everything else calm; respect prefers-reduced-motion.

Suitable technologies

- Framer Motion — declarative, easy-to-scope animation for the signature moment
 - CSS @media (prefers-reduced-motion) — accessibility-respecting fallback
-

Phase 6 — Responsiveness & Accessibility Polish

- **Goal:** Support the real-world setup: a tablet or laptop propped beside a notebook.

Agent task prompt

Make the app work well on a tablet propped next to a notebook — the primary real-world use case, not just desktop. Ensure visible keyboard focus states, sufficient contrast, and readability at arm's length.

Suitable technologies

- Tailwind responsive breakpoints — tablet/laptop layout adjustments
 - axe DevTools / Lighthouse accessibility audit — contrast and focus-state checks
-

Phase 7 — Edge Cases & Correction

- **Goal:** Handle the messy inputs students actually have.

Agent task prompt

Handle multi-page PDFs, rotated or skewed photos, poor-lighting OCR failures, and long documents. Add an inline text-correction box since OCR is never 100% accurate.

Suitable technologies

- PDF.js multi-page iteration — looping OCR/extraction across pages
 - browser-image-compression + canvas rotate — basic photo pre-processing before OCR
 - controlled `<textarea>` — inline manual text correction
-

Phase 8 — Test & Ship

- **Goal:** Confirm the full flow works end-to-end, then deploy.

Agent task prompt

Use Antigravity's browser-control agent to auto-test the upload → OCR → TTS flow end-to-end on a few real PDFs and photos. Then deploy to Vercel or Netlify.

Suitable technologies

- Antigravity browser-agent — automated end-to-end flow testing
- Vercel or Netlify — one-click static hosting/deploy for the Vite build